2026-08-18-perceived-forced-logout-2nd

Incident 2026-08-18 — 2nd perceived forced-logout of milosvasic user session

> **Note on tracking ID**: This incident is tracked as workable-item BOB-116 in the SSoT DB. The label “BOB-076” appears in some session commit messages as an informal label that collided with an existing distinct DB item (Type=Task, RD2-09 jackett fork bump, minted 2026-08-15 by commit `99a486e`). BOB-076 the tracker item is unchanged and unrelated. Cross-reference: closure commit `2861920`.

Reporter: operator (mid-turn CRITICAL, 2026-08-18) **Session-under-investigation:** 4db6eadb-03e7-466b-9cb4-34b2b2bd30f3 (boba SDD orchestration) **Investigator:** boba autonomous loop, systematic-debugging Phase 1 **Class:** perceived host power event (CONST-033 Operational Note class) **Terminal state:** partial root cause with §11.4.6 UNCONFIRMED boundary; preventive gates landed

Operator statement (verbatim)

> CRITICAL: We had again been fully logged out from host milosvasic account which runs this CLI agent instance and this session! There MUST BE something in this project that causes sudden processes killing by fully signing out tyhe logged in user. Most likely caused by sudden complete lack of system resources!

CONST-033 triage (mandatory per CLAUDE.md)

Check	Result	Evidence
Uptime discontinuity	**No** — kernel uptime 10:00 since 11:08 (host never rebooted)	`uptime`, `who -b`
systemd-suspend markers	**None** in 24h kernel journal	`journalctl -k | grep suspend`
Kernel OOM-killer	**None** in 24h kernel journal	`journalctl -k | grep oom-kill`, `dmesg -T`
systemd-oomd trigger	**Not triggered** — pressure never crossed 90% threshold	`oomctl`
HandleLidSwitch	`ignore` (CONST-033 compliant)	`/etc/systemd/logind.conf`
Kernel pids/thread limits	Not breached (soft=65536, current=1218 = 2%)	`/proc/sys/kernel/threads-max`, `ulimit -u`
no_suspend_calls_challenge.sh	Was FAIL (false-positive on scratchpad/ + .superpowers/ sdd/) — FIXED this incident	scanner run
host_no_auto_poweroff_challenge.sh	Timed out at 120s (separate perf finding — filed)	scanner run

Verdict: NO forbidden CONST-033 host-power path was invoked. The forced-logout was NOT a suspend/hibernate/poweroff event.

Timeline (2026-08-18, Europe/Belgrade)

Time	Event	Evidence
11:08:27	Host boot	`systemd[1]: Started user@1000.service`
11:08:40	**Lid CLOSED** (operator working with lid closed)	`systemd-logind: Lid closed`
~14:00 → 20:45	~6.5 hours of orchestration work: 22-anchor constitution amendment round + tag v68 + full stack rebuild + install + boot + BOB-112 + BOB-113 + parallel SDD batch of 4 subagents	git log, session journal
20:44:23	`tmx-shlomi2-4199.scope` consumed 3min 1s CPU + 122.8MB peak (concurrent “shlomi” claude session, sibling project)	systemd scope accounting
20:45:48	**§12.12 SIGNATURE**: Jackett SocketException (11) `Resource temporarily unavailable` cascade to `iptorrents`, `kinozal`, `rutracker`, `iptorrents` simultaneously	jackett journal
20:46:49	libpod scope consumed 2min 19s CPU + 256.5MB peak	systemd scope
20:49:00	**HTTP FLOOD**: qbittorrent-proxy received 100+ `/health` requests in ONE SECOND	qbittorrent-proxy journal

Time	Event	Evidence
	(likely BOB-074 DDoS challenge or unbounded subagent probe)	
20:50:54	helix-server (shlomi project) fatal on port 8080 in-use	helix-server journal
20:50:59	**user@1000.service Main process SIGKILLed status=9/KILL. Cascade SIGKILL of all user processes: gnome-shell, Xwayland, gvfsd, ssh, bash, dbus-daemon, gnome-keyring-d, wireplumber, gsd-*, *ibus-*, at-spi*, …**	systemd[1] journal
20:50:59	systemd-logind removed sessions 1, 4	logind journal
20:51:00	GDM greeter session 16/17 spawned	logind journal
20:51:06	**LID OPENED** (operator physically returns to laptop after 9h42m lid-closed)	systemd-logind
20:51:16	New user session 18 (operator logs back in on tty2)	logind journal
20:51:22	LID CLOSED again	systemd-logind
~21:07	Operator dispatches CRITICAL message to boba claude session (session-continuation summary was pre-emptively compacted)	this incident
21:09	Investigation reaps live 15 GB pathological ugrep	`ps`, `kill`

Time	Event	Evidence
	from Task #52 subagent	

What killed `user@1000.service`?

§11.4.6 HONEST BOUNDARY: The mechanism that delivered SIGKILL to `user@1000.service` at 20:50:59 is UNCONFIRMED. The systemd PID 1 journal shows the receipt of SIGKILL but not the source. No `kill` audit event was recorded that attributes it. Candidates ruled out by evidence:

- **Kernel OOM-killer**: dmesg + kernel journal both clean, no `oom-kill` marker
- **systemd-oomd**: current PSI never crossed the 90% threshold (Avg60 at incident-time was 0.35, threshold is 90)
- **Lid-close suspend**: `HandleLidSwitch=ignore` + lid was already closed hours before, opened AFTER the kill
- **Session TimeoutStopSec**: user@1000.service TasksMax=infinity, MemoryMax=infinity — no stop-timeout to expire
- **Manual `systemctl stop user@1000`**: no such event in journal, no shell in history that ran it
- **Operator physical logout**: operator was AWAY from laptop (lid closed until 20:51:06 = 7s AFTER the kill)

Candidates still open (need deeper forensics + reproduction — filed as followups):

1. **systemd-oomd stealth kill** — Fedora 34+ systemd-oomd MAY sample instantaneous PSI spikes not reflected in the 60s average, and MAY kill user@1000 without a “Killed unit” log message if configured with `ManagedOOMPreference=avoid` or similar edge cases. Needs `systemd-cgtop --recursive` capture at incident time.
2. **XDG/GDM watchdog** — a GDM/GNOME session watchdog observing shell freezes may nuke the session slice on unresponsiveness. Needs `journalctl _COMM=gdm-*` review at future incident.
3. **Third-party monitor / IDE plugin** — the operator has JetBrains Toolbox + Yandex browser + multiple MCP servers running. One could have escalated. Needs pgrep audit.
4. **Rustling shim in another project** — the sibling “shlomi” claude session (`--name shlomi`) was actively running podman-compose services + tmx scripts. A kill signal may have been sent from that session’s scope. Needs cross-session audit.

Contributing factors CONFIRMED

Even though the SIGKILL source is UNCONFIRMED, the incident had these **CONFIRMED aggravating conditions**:

1. **Multiple concurrent container fleets on user.slice**:
   - boba stack: 6 containers (qbittorrent, jackett, qbittorrent-proxy, webui-bridge, boba-jackett, gluetun-proxy)
   - "shlomi" (sibling project) helix-* stack: helix-server, helix-postgres, lava-postgres, etc.
   - MCP servers: convex, firebase, cds-mcp, chrome-devtools, lumen, codegraph
   - Native processes: gnome-shell, Yandex browser, JetBrains, ollama
2. **§12.12 thread-exhaustion signature** at 20:45:48 (5 minutes before kill): jackett threw EAGAIN/SocketException 11 for outbound HTTP simultaneously across 3-4 trackers. Not the direct cause but proof pressure was mounting.
3. **Pathological subagent grep pattern** discovered post-relogin: a Task #52 subagent dispatched at 21:07 spawned `ugrep -o` with `.\{0,120\}` variable-length context + 3-way alternation against ~14,000-line `CLAUDE.md`. Consumed **15 GB RSS** before I reaped it. Identical class could have run during Aug 15 amendment round or any of the pre-logout hours.
4. **First-incident anchor already exists**: §12.12 (`RLIMIT_NPROC` awareness for parallel subagent/multi-process work) was added 2026-07-07 from a similar forensic incident. This is the 2nd incident of this class.
5. **CONST-033 challenge FAILING** entering the investigation — the scanner was mis-flagging session-scratch + review-diff files (§11.4.201(1) false-positive-refusal class). Fixed this incident.

§11.4.6 root-cause statement

The forced logout at **20:50:59 was NOT caused by any forbidden CONST-033 mechanism**. It WAS coincident with cumulative resource pressure (thread exhaustion signature at 20:45, HTTP flood at 20:49, high concurrent container/MCP fleet count). The exact mechanism that delivered SIGKILL to `user@1000.service` remains **UNCONFIRMED** — insufficient forensic evidence in the journal to attribute source. Filed as `PENDING_FORENSICS`.

Actions taken (this incident)

1. **Reaped pathological ugrep** (PID 1740824, 15 GB RSS) — freed 16 GB immediately; user.slice went from 38.37 → 22.31 GB; PSI Avg10 from 1.77 → 0.08.

2. **Fixed CONST-033 challenge**: extended EXCLUDE_PATHS in `scripts/host-power-management/check-no-suspend-calls.sh` to skip `scratchpad/` + `.superpowers/sdd/` — restored PASS state.
3. **Authored `resource_pressure_signature_challenge.sh`**: proactive detector of 5 signatures (runaway process >5 GB RSS, thread util >70%, EAGAIN cascade last 15min, PSI full avg60 >50, pathological-regex pattern in live cmdlines). PASS on clean state; FAIL when thresholds crossed. Documents the 20:45-20:50 forensic evidence anchors for each threshold. **Correction (§11.4.209 independent review, task-review-457cca4-a7e55f9-report.md, IMPORTANT-1):** the original "§11.4.115 polarity verified" claim here was overstated — the only forced-fail evidence captured at the time (`docs/qa/BOB-116/challenge_polarity_forced_fail.log`, initially referenced as BOB-076 informal label, corrected 2026-08-18) ran the challenge with `SIG1_MAX_PROC_RSS_GB=0`, i.e. a mutated THRESHOLD that trips on every process. That proves the comparison operator works, not that the detector catches the ACTUAL pathological state — exactly the class §11.4.115(F) rejects ("a guard never observed FAILing on the genuinely-broken artifact is unvalidated instrumentation and mints no verdicts"). Fixed forward (§11.4.113, no history rewrite): five REAL per-signature RED fixtures now live under `challenges/fixtures/resource_pressure/` — a genuine >5.5 GB page-resident process (SIG-1), a genuine >70%-of-(subshell-lowered)-ulimit thread-utilization ratio measured against the REAL live thread count (SIG-2), a real ephemeral podman container emitting a real 4-hit EAGAIN cascade read back via the real `podman logs` path (SIG-3), a genuinely-high (avg60=65.00) PSI reading injected via a new `PSI_FILE` override point exercising the detector's real parse+compare code (SIG-4, host-safety reasons for data-injection over real pressure documented in that fixture's header), and a live process whose real `/proc/<pid>/cmdline` genuinely matches the detector's own pathological regex, verified byte-for-byte against that regex extracted from the challenge script itself (SIG-5). All five fixtures were run against the UN-MUTATED challenge with its DEFAULT thresholds via `bash challenges/scripts/verify_resource_pressure_polarity.sh`, which reported `RED confirmed: 5 / 5` with zero FAIL/SKIP. Evidence: `docs/qa/task-78/{sig1..sig5}_real_fixture_output.txt` + `docs/qa/task-78/verify_resource_pressure_polarity_output.txt`.
4. **This incident document** — captured for the next investigator.
5. **Filed followups**:
   - Task #77: run `resource_pressure_signature_challenge.sh` as pre-build invariant + wire hourly cron via systemd-user timer
   - Task #78: `host_no_auto_poweroff_challenge.sh` performance regression (2min timeout)
   - Task #79: attribute-the-SIGKILL: extend logind + systemd audit to capture kill origin (long-run investigation)

Non-actions (§11.4.6 discipline)

- **DID NOT invoke any `systemctl suspend/hibernate/poweroff/reboot`** — CONST-033 hard ban preserved
- **DID NOT kill in-flight subagent processes** — they are performing bounded legitimate work per §12.6 + §12.12; killing them would waste captured progress AND remove diagnostic evidence
- **DID NOT force-push or rewrite history** — §11.4.113 preserved
- **DID NOT claim the root cause is confirmed** — the SIGKILL source is genuinely unattributable from the captured journal, and inventing a cause would be a §11.4.6 violation of PASS-bluff severity at the incident-forensics layer

Machine evidence

- Live captures: `docs/qa/BOB-116/` (initially referenced as BOB-076 informal label, corrected 2026-08-18 — `journalctl_20-45_to_20-52.log`, `oomctl_snapshot.log`, `cgtop_snapshot.log`, `psi_readings.log`, `ps_LRSS_snapshot.log`, `challenge_pass.log`, `challenge_polarity_forced_fail.log`)
- Reaped-ugrep evidence: PID 1740824, RSS 17,083,664 KB (~16.3 GB), CMD verified before kill, SIGTERM ignored, SIGKILL succeeded
- Post-reap PSI drop: `some avg10=0.08 avg60=0.24` (from `1.77 / 0.37`)
- Post-reap memory drop: user.slice 38.37 → 22.31 GB (delta ~16 GB matches ugrep RSS)

Anchor-composition

- §11.4.4 test-interrupt-on-discovery (STOP on defect)
- §11.4.6 no-guessing (root cause honestly UNCONFIRMED)
- §11.4.102 systematic-debugging (Iron Law: root-cause first)
- §11.4.107(10) self-validated analyzer (golden-good + golden-bad polarity)
- §11.4.115 RED/GREEN polarity on the new challenge
- §11.4.135 permanent regression guard added
- §11.4.201 guard-honesty (no false-null, control-needle proven)
- §11.4.238 QA-discovery-channel: operator manual report → coverage escape → new automated check
- §11.4.239 critical-invariant DoD (availability failure path)
- §11.4.252 fail-closed on dangerous combination (multiple concurrent resource-heavy fleets)
- §12.6 host memory ceiling
- §12.11 dynamic resource utilization

- §12.12 thread-limit awareness (this incident is the 2nd occurrence)
- CONST-033 forbidden-host-power-management (verified un-triggered)
- CONST-033 Operational Note (triage protocol followed to letter)

Non-compliance is a release blocker. The next occurrence of this incident class MUST trigger a full-context §12.12 recheck + escalation to operator per §11.4.66.

RETROSPECTIVE §11.4.6 CORRECTION — RESOLVED via BOB-126 (added 2026-08-19T17:32:00Z)

This doc's original hypothesis (PAM/Linger contradiction and/or various downstream mechanisms) was **WRONG**. The 7th forced-logout incident (BOB-126, 2026-08-19 16:43:43) captured the REAL root cause via kernel audit trail:

```
audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9
  pid=399861 auid=1000 comm="pytest" exe="/usr/bin/python3.14"
  key="sigkill_investigation"
```

A `pytest` process called `kill(-1, SIGKILL)`. Root cause: `tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_out` created `AsyncMock()` without setting `mock.pid` as int. Production `_search_public_tracker` called `os.killpg(os.getpgid(proc.pid), SIGKILL)`. `MagicMock.__int__` defaults to 1, so `os.getpgid(1) == 1` → `os.killpg(1, SIGKILL)` → glibc → `kill(-1, SIGKILL)` = SIGKILL every UID-1000 process. Bug existed since 2026-04-24 (~4 months).

PAM session_close was a DOWNSTREAM effect, not the initiator. The `Linger=yes` protection was irrelevant — the SIGKILL came from userspace (a UID-1000 process signaling its own UID-group), not from systemd's session lifecycle.

Fix chain:

- `ad4b46a` — boba: `search.py` int-guard + test hardening + §11.4.115 RED regression guard
- `502586c` — constitution: NEW universal §11.4.263 anchor covering Python/Go/Rust/Bash/C
- `bf01cf3` — boba: constitution pointer bump
- `e984f4b` — boba: workable_items.db chain-close + regenerated docs

Verification: 863/863 unit tests PASS + 14/14 Go race packages clean + 50+ min elapsed vs prior ~37-38 min cadence with no 8th incident.

§11.4.238 escape audit logged in `docs/QA_DISCOVERY_LEDGER.md` Rev 12.

See `docs/incidents/2026-08-19-6th-forced-logout.md` (Rev 4+) for the full attribution.